

Specyfikacja Projektu Zaliczeniowego (Programowanie w Języku Java)

Markiian Novytskyi, Arseni Fayeau

12 kwietnia 2026

1. Tytuł Projektu

Java Royale: Multiplayer Top-Down Battle Royale Shooter

2. Opis i Cele Projektu

Projekt polega na stworzeniu wieloosobowej gry akcji typu "Battle Royale" z widokiem z góry (top-down). Aplikacja oparta jest na architekturze klient-serwer z modelem serwera autorytatywnego, co zapewnia pełną synchronizację stanu gry oraz uczciwość rozgrywki.

Główne cele projektu:

- Implementacja wydajnego serwera gry opartego na bibliotece **Netty**, obsługującego komunikację asynchroniczną.
- Stworzenie wydajnego klienta graficznego w **libGDX** wykorzystującego sprzętową akcelerację OpenGL.
- Synchronizacja stanu wielu graczy w czasie rzeczywistym (ruch, strzelanie, zdrowie).
- Implementacja mechaniki zmniejszającej się "bezpiecznej strefy" wymuszającej interakcję między graczami.

3. Wymagania Funkcjonalne

- **System Lobby:** Łączenie się z serwerem i oczekiwanie na minimalną liczbę graczy.
- **Mechanika Ruchu:** Poruszanie się postacią (WSAD) z predykcją po stronie klienta.
- **System Walki:** Strzelanie pociskami w kierunku kursora myszy i rejestracja trafień na serwerze.
- **Synchronizacja Świata:** Widoczność innych graczy i ich działań w czasie rzeczywistym.
- **Mechanika Strefy:** Dynamicznie zmniejszający się obszar gry zadający obrażenia graczom poza nim.
- **Warunki Zwycięstwa:** Wyłonienie ostatniego żywego gracza i zakończenie sesji gry.

4. Wymagania Niefunkcjonalne

- **Wydażność:** Rendering po stronie klienta na poziomie stabilnych 60 FPS.
- **Skalowalność:** Architektura pozwalająca na jednoczesną obsługę do 25 (50) graczy na jednej mapie.
- **Niskie Opóźnienia:** Optymalizacja protokołu sieciowego w celu minimalizacji lagów.
- **Stabilność:** Serwer odporny na gwałtowne rozłączenia użytkowników.
- **Modularność:** Podział na moduły Maven (`common`, `server`, `client`).

5. Architektura Systemu

Typ architektury: **Klient-Serwer (Authoritative Server Model)**.

Schemat modułów:

- **Common:** Wspólne klasy pakietów (`Packets`), modele danych (`PlayerState`) i stałe gry.
- **Server:** Logika gry (silnik fizyczny, kolizje, zarządzanie strefą), brak interfejsu graficznego.
- **Client:** Renderowanie obrazu (`libGDX`), obsługa wejścia (mysz/klawiatura), komunikacja Netty Client.
- **Networking:** Wykorzystanie biblioteki Netty do przesyłania zserializowanych obiektów przez TCP.

6. Zastosowane Algorytmy

- **Interpolacja Liniowa (LERP):** Wygładzanie ruchu graczy na ekranie w celu wyeliminowania przeskoków wynikających z opóźnień sieciowych.
- **Detekcja kolizji:** Sprawdzanie trafień pocisków w graczy oraz kolizji postaci z przeszkodami na mapie.
- **Algorytm bezpiecznej strefy:** Obliczanie dystansu euklidesowego gracza od środka strefy w celu zadawania obrażeń.

7. Struktura Klas

Moduł Common

- `GamePacket` – klasa bazowa dla komunikatów sieciowych.
- `PlayerMovePacket` – dane o pozycji i rotacji gracza.
- `PlayerState` – model przechowujący zdrowie, pozycję i statystyki.

Moduł Server

- **ServerLauncher** – inicjalizacja **ServerBootstrap** i pipeline’u Netty.
- **WorldManager** – główna pętla gry po stronie serwera (tick-rate).
- **ServerHandler** – obsługa przychodzących pakietów i rozsyłanie stanu do klientów.

Moduł Client

- **ClientLauncher** – konfiguracja okna LWJGL3 i start aplikacji.
- **GameClient** – główna pętla renderowania libGDX.
- **NetworkManager** – zarządzanie wątkiem sieciowym Netty Client.

8. Wykorzystane Technologie i Biblioteki

Język i Narzędzia:

Java 21, Maven (Multi-module Project).

Biblioteki:

- **Netty 4.1:** Framework sieciowy.
- **libGDX 1.12.1:** Silnik graficzny (OpenGL).
- **JUnit 5:** Testy jednostkowe logiki wspólnej.

9. API i Modele Danych

Wymiana danych odbywa się poprzez asynchroniczne pakiety.

Przykładowy Model Pakietu (JSON-like representation):

```
{
  "playerId": "int",
  "x": "float",
  "y": "float",
  "rotation": "float",
  "isShooting": "boolean"
}
```

10. Interfejs Użytkownika

- **Ekran Główny:** Proste menu z możliwością podania nicku i połączenia z serwerem.
- **Ekran Gry:** Główne okno renderujące mapę, graczy i pociski.

- **HUD:** Wyświetlanie paska zdrowia, liczby pozostałych graczy oraz czasu do zmniejszenia strefy.

11. Etapy Realizacji

Faza	Opis	Czas
Faza 1	Konfiguracja projektu Maven, struktury modułów i Git	1 tydzień
Faza 2	Implementacja serwera Netty i bazowej komunikacji	2 tygodnie
Faza 3	Budowa silnika graficznego w libGDX (rendering, wejście)	2 tygodnie
Faza 4	Synchronizacja fizyki i walki na serwerze autorytatywnym	1 tydzień
Faza 5	Testy, optymalizacja lagów i poprawki błędów	1 tydzień

12. Testowanie

- **Testy Jednostkowe:** Walidacja algorytmów kolizji i matematyki w module `common`.
- **Stress Tests:** Uruchomienie wielu klientów na jednej maszynie w celu sprawdzenia wydajności serwera.
- **Testy Integracyjne:** Sprawdzenie poprawności przepływu pakietu od kliknięcia do rozesłania przez serwer.

13. Ryzyka i Środki Zaradcze

Ryzyko	Wpływ	Zarządzanie
Problemy z synchronizacją (lagi)	Wysoki	Implementacja interpolacji i predykcji.
Złożoność fizyki multiplayer	Wysoki	Uproszczenie modeli kolizji do okręgów i prostokątów.

14. Kryteria Sukcesu

- Płynna rozgrywka dla minimum 10 graczy jednocześnie.
- Poprawna rejestracja trafień i redukcja punktów życia.
- Stabilne działanie serwera przez minimum 30 minut sesji.

15. Szacunkowe Nakłady Pracy

Całkowity czas: **155 godzin**.

Moduł / Zadanie	Opis	Czas (h)
Architektura i Maven	Setup modułów i zależności	10
Networking (Netty)	Server/Client handlers	30
Rendering (libGDX)	SpriteBatch, Camera, Input	45
Logika Gry (Physics)	Kolizje, Strefa, Strzelanie	30
UI / HUD	Menu, paski życia	15
Testowanie i Debugging	Poprawa synchronizacji	25